Doksli Lore InterupsiArOS

Untuk kalangan kami saja



Kelompok InterupsiArOS

Dipta	Raja Pertama
Bagas	Raja Kedua
Aqsha	Raja Ketiga
Sultan	Raja Terakhir

Sekolah Teknik Elektro dan Informatika - Komputasi

Institut Teknologi Bandung

2026/2027

Chapter 1

PENDAHULUAN

Dunia ini penuh dengan kegelapan, kebodohan, dan kepengecutan. Nilai-nilai inheren yang tumbuh di jiwa manusia sejak eksistensi mereka dihembuskan oleh Prometheus, jiwa-jiwa pejuang yang haus akan kemenangan dan tak tergoyahkan sekalipun oleh kehancuran tempat tinggalnya dan jatuhnya jasad rekan sejati mereka di medan perang, telah hilang. Kini, manusia berada di titik terendah spesies mereka. Sekumpulan organisme yang seharusnya merupakan puncak rantai makanan dari segala makhluk yang pernah hidup di muka bumi telah kehilangan status mereka sebagai entitas terkuat di bawah para dewa-dewi, menjadi makhluk terhina, yang eksistensinya merupakan pengkhianatan terhadap para dewa-dewi itu sendiri.

Itulah kata-kata yang keluar dari mulut Ares, Sang Dewa Perang yang Bengis dan Haus akan Kemenangan, namun Bijak dan Pendorong Keberanian. Kata-kata tersebut bukanlah sebuah kekecewaan semata, melainkan sebuah frustrasi yang diakumulasi oleh sang Dewa sejak kejatuhan masyarakat Yunani. Kata-kata tersebut adalah peringatan yang nyata bagi umat manusia di seluruh dunia.

Kami, 4 Raja Pejuang Agung, adalah titisan para SPARTANS yang senantiasa berjuang demi bara api perjuangan yang menuntut akal yang menakar, hati yang merasa, serta perjalanan yang panjang di depan mata para pejuang sejati. Kami adalah representasi dari Ares yang akan membawa semangatnya ke dunia fana ini. Kami membutuhkan sesuatu, agar nilai-nilai Ares dapat tersampaikan secara luas. Oleh karena itu, tertulis secara mentah pada surat ini, kami mengorbankan segala darah, air mata, dan keringat dalam penciptaan sebuah mahakarya kami yang dapat menciptakan tether kompatibilitas bagi Ares di dunia yang hina ini serta menyambungkan titah-titah Ares pada sinapsis seluruh umat manusia di muka Bumi ini baik yang masih hidup maupun sudah tiada. Melalui InterupsiArOS, kami akan menghidupkan kembali jiwa-jiwa semangat perang dan urat keberanian manusia dan membawa mereka kembali ke medan perang kami, para SPARTANS.

Tidak ada ruang untuk kelemahan dan ketidaksiapan di sini. Yang akan hidup hanyalah mereka yang dapat beradaptasi, memiliki gairah dan kekuatan untuk mengekspansikan kekuasaan bagai ekspansi kofaktor pada sebuah operasi invers matriks, serta mereka yang mempersiapkan diri untuk perjalanan yang tidak pasti.

Chapter 2

TOOLCHAIN, KERNEL, & GDT

2.1 How an OS starts up

Bagaimana booting sebuah OS komputer sebenarnya terjadi? Secara singkat, jawabannya adalah sebagai berikut:

1. Listrik mengalir ke hardware
2. CPU menjalankan firmware BIOS (Basic Input/Output System) yang terdapat di motherboard
3. BIOS menjalankan POST (Power-On Self Test) yang melakukan pengecekan terhadap fungsionalitas hardware
4. Jika POST berhasil, BIOS akan memuat MBR (Master Boot Record), yaitu sektor 512 bytes pertama pada bootable device (penyimpanan yang mengandung OS), ke RAM
5. MBR menjalankan kode Bootstrap, lalu Bootstrap memindahkan Bootloader dari bootable device ke RAM dan mengeksekusinya
6. Bootloader memindahkan OS ke RAM sehingga OS akhirnya dapat digunakan

Fungsinya bisa disimpulkan sendiri dari nama-namanya.

2.2 File-file awal aneh

File-file ini disediaiin sedari awal.

2.2.1 grub1

File grub1 yang terletak di other/grub1 adalah bootloader yang dipakai di Tubes OS ini.

2.2.2 kernel-entrypoint.s

Kernel itu apa ya kak? Kernel itu adalah inti dari sebuah OS yang menjalankan hal-hal seperti syscall, manajemen memori, scheduling, dll. Basically kernel adalah core dari penyambung interface dengan hardware.

Perlu diketahui bahwa kernel tidak bisa langsung membaca bahasa C, namun harus melalui kode Assembly terlebih dahulu, yang dalam kasus ini adalah kernel-entrypoint.s itu sendiri (dibantu loader juga, tapi itu nanti).

Secara umum, yang dilakukan oleh kernel-entrypoint.s adalah set-up agar loader dan GRUB bisa melakukan pekerjaannya. kernel-entrypoint.s menyediakan Call Stack juga agar bahasa C dapat digunakan. kernel-entrypoint.s juga melakukan load_gdt.

2.2.3 kernel-entrypoint.h

Not to be confused with kernel-entrypoint.s, tapi ini header file buat eksekusi load_gdt.

2.2.4 gdt.h

Header untuk... GDT. Nanti dibahas lagi.

2.2.5 string.h & string.c

Skip dulu untuk saat ini. I think belum kepake. But singkatnya manipulasi memory like compare, copy, move, write.

2.2.6 portio.h & portio.c

Singkatnya untuk akses port x86 menggunakan assembly.

2.2.7 intsetup.s

Source code ASM buat interrupt thingy.

2.2.8 Makefile

Ya basically buat automate compiling and linking and all that aja sih.

2.3 Kernel and Linker

2.3.1 kernel.c

Basically main programnya. Atur-atur lah.

2.3.2 linker.ld

Basically yang melakukan menggabungkan objek hasil kompilasi jadi satu binary executable. Entry dilakukan dari loader. Kode di-run dengan batas bawah 0x00100000 (1 MB) serta aligning by 4 KB lots of things like:

- .multiboot
  Atur-atur lah, multiboot header supaya kernel bisa dicari sama GRUB
- .text
  Menyimpan kode program
- .rodata
  Menyimpan read only
- .data
  Menyimpan global variabel
- .bss
  Menyimpan uninitialized variables

2.4 Buat apa Makefile?

Seperti yang dijelaskan sebelumnya, fungsi Makefile adalah untuk melakukan kompilasi dan linking serta as a whole mengotomasi proses building proyek.

```
ASM             = nasm

UNAME_S         := $(shell uname -s)
ifeq ($(UNAME_S),Darwin)
LLVM_PREFIX     := $(shell brew --prefix llvm)
CC              = clang
LIN             = $(shell brew --prefix lld)/bin/ld.lld
II              = mkisofs
ARCH_CFLAG      = --target=i386-elf -march=i386 -mno-mmx -mno-
    sse -mno-sse2 -mno-avx -msoft-float -Wno-unused-command-
    line-argument
```

```
else
CC              = gcc
LIN             = ld
II              = genisoimage
ARCH_CFLAG      =
endif
```

Basically ini buat setup compiler, linker, dan tool genisoimage. Dibedakan untuk macOS dan Linux yang bisa dilihat dari ifeq dan else-nya.

```
SOURCE_FOLDER = src
OUTPUT_FOLDER = bin
ISO_NAME      = OS2025
```

Ini buat lokasi folder source, output, dan file name iso yang ingin dibuat

```
WARNING_CFLAG = -Wall -Wextra -Werror
DEBUG_CFLAG   = -fshort-wchar -g
STRIP_CFLAG   = -nostdlib -fno-stack-protector -nostartfiles
    -nodefaultlibs -ffreestanding
CFLAGS        = $(DEBUG_CFLAG) $(WARNING_CFLAG) $(STRIP_CFLAG
    ) $(ARCH_CFLAG) -fno-builtin -m32 -c -I$(SOURCE_FOLDER)
AFLAGS        = -f elf32 -g -F dwarf
LFLAGS        = -T $(SOURCE_FOLDER)/linker.ld -melf_i386
IFLAGS        = -R -b boot/grub/grub1 -no-emul-boot -boot-
    load-size 4 -A os -input-charset utf8 -quiet -boot-info-
    table
```

CFLAGS buat GCC, AFLAGS buat ASM, LFLAGS buat Linker, IFLAGS buat ISO maker. Probably ga perlu terlalu paham terlalu banyak terkait flags-flagsnya.

```
run: all
        @qemu-system-i386 -s -S -cdrom $(OUTPUT_FOLDER)/$(
            ISO_NAME).iso
all: build
build: iso
clean:
        rm -rf $(OUTPUT_FOLDER)/*.o $(OUTPUT_FOLDER)/*.iso $(
            OUTPUT_FOLDER)/kernel
```

`run` untuk `build` dan run OS dari ISO pakai qemu. `all` and `build` harusnya melakukan hal yang sama, yaitu membuat disk image ISO, also does `kernel`. `kernel` buat ngurus ASM, GCC, dan Linker. `clean` buat bersihin file output.

PASTIKAN SETIAP C FILE BARU TAMBAHIN DI COMPILE KERNEL!!!

2.5 Global Descriptor Table (GDT)

Basically, GDT adalah tabel yang berisi Segment Descriptor. Segment Descriptor ini adalah tipe data yang menyimpan informasi-informasi tentang suatu segmen, which is daerah memori linier. Informasi yang disimpan terutama adalah flag-flag dan tingkat hak akses tertentu. Di proyek ini, kita pakai Descriptor Privilege Level (DPL) Ring 0 untuk Kernel Mode dan Ring untuk User Mode.

2.5.1 gdt.h dan gdt.c (gabung aja lah ya)

- `GDT_MAX_ENTRY_COUNT 32`
  Ukuran maksimal Segment Descriptor dalam suatu GDT

- `GDT_KERNEL_CODE_SEGMENT_SELECTOR`
  Indeks Segment Tipe Code (nanti dijelasin), whichis Indeks Pertama whichis sizeof(SegmentDescriptor) whichis 8 bytes. 8 bytes kali 1 = 0x8

- `GDT_KERNEL_DATA_SEGMENT_SELECTOR`
  Indeks Segment Tipe Data (nanti dijelasin) whichis indeks kedua jadi 0x10 aka 16.

- `SegmentDescriptor`
  Struct representasi SegmentDescriptor. SegmentDescriptor berukuran 8 byte. Bisa dilihat memanfaatkan kemampuan C untuk menyesuaikan ukuran bit suatu tipe data (sintaks bit-field). Isi SegmentDescriptor dideskripsikan in a bit. Also notice, di struct ini dan beberapa struct lainnya ada ___attribute___((packed)). Funginya adalah supaya ga ditambahin padding of any sorts jadi ukurannya emang udah fix segitu.

- `GlobalDescriptorTable`
  Memuat array berukuran entri maksimum tipe data SegmentDescriptor

- `GDTR`
  Basically register yang memuat informasi terkait GDT supaya bisa dicari CPU di RAM. Memuat ukuran GDT dan alamatnya. Note ada extern struct GDTR bernama ___gdt_gdtr, basically dibuat variabel global shared lah. Sizenya sizeof gdt dikurangi 1

Gambarnya liat sendiri di Buku dan Intel Manual.
Jika kita bagi dari segi definisi dan fungsinya, ada bagian-bagian berikut:

- Segment Limit Field
  Ukuran dari segment. Diwakili oleh segment_low di bit 0-15 di 16 byte pertama

dan segment_high di bit ... itung sendiri udah gede. Ukuran juga itung sendiri be.

- Base Address Field
  Menyatakan alamat fisik awal dari lokasi segmen. Diwakili oleh base_low, base_mid, dan base_high.

- Type Field
  Tipe dan hak akses segmen. Misalnya executable (code), readable, atau writable. Diwakili oleh type_bit.

- S (Descriptor Type) Flag
  Jika 1, segmennya kode/data. Jika 0, segmennya sistem. Karena di kita kode/data semua, jadi dinamain non_system.

- DPL Flag
  Ring-ring DPL-nya. Diwakili oleh privilege.

- P (Present) Flag Jika segmen ada di RAM, nilainya 1. Diwakili oleh valid_bit.

- AVL (Available and Reserved Bit) Flag
  Available bit, sesuaikan saja SPARTANS. Diwakili avl.

- L (64-Bit Code) Flag
  Digunakan pada arsitektur x64. Di kita, nilainya 0 karena kita bikin sistem x86. Diwakili long_mode.

- D/B (Default Operation Size/Default Stack Pointer And/Or Upper Bound) Flag
  Basically ngatur ukuran operasi (executable segment), ukuran stack pointer, dan upper bound segment. Nilainya 1 jika 32-bit system, 0 jika 16-bit. Diwakili oleh opr_32_bit.

- G (Granularity) Flag
  Scaling segment limit. Kalo 0, pake unit per byte jadi maks 1 MB. Kalo 1, per 4KB jadi maks 4GB. Diwakili oleh granularity.

Terlihat susunannya seperti di bawah.

```
// First 32-bit
uint16_t segment_low;
uint16_t base_low;

// Next 16-bit (Bit 32 to 47)
uint8_t base_mid;
```

```
    uint8_t type_bit    : 4;
    uint8_t non_system  : 1;
    uint8_t privilege   : 2;
    uint8_t valid_bit   : 1;

    uint8_t segment_high : 4;
    uint8_t avl          : 1;
    uint8_t long_mode    : 1;
    uint8_t opr_32_bit   : 1;
    uint8_t granularity  : 1;
    uint8_t base_high;
```

2.5.2 Load GDT

Di kernel.c, kita cukup menambahkan:

```
void kernel_setup(void) {
    // ...
    load_gdt(&_gdt_gdtr);
    // ...
}
```

2.5.3 Bonusan aja sih

Di gdt.c dan possibly beberapa file ke depan, kita pakai yang namanya Designated Initializers (pokoknya depannya pakai titik). Teknik ini dipakai supaya langsung mengacu ke variabel di struct pada posisi address yang bersesuaian terlepas dari urutan kita menginputnya.

Sesuai tabel GDT yang kita buat, indeks ke-0 diisi GDT null, indeks ke-1 buat segmen Kernel Code, indeks ke-2 buat segmen Kernel Data.

Aturan-aturannya bisa dilihat sendiri di buku.

Kernel Code bersifat Read-only and Executable dan berfungsi menyimpan instruksi binary yang dikompilasi. Kernel Data bersifat Read-write and Non-executable dan berfungsi menyimpan variabel global, stack kernel, dan system-wide structures.

Ya buzzword buzzword dikit. Tapi FYI aja.

2.6 Awal Mula Kebangkitan Ares

Dunia akan digetarkan oleh prosedur-prosedur yang kami buat demi cinta kami kepada kota kami terdahulu, SPARTA. Persiapan kami dalam membangkitkan kembali

mimpi-mimpi mulianya akan memberikan esensi derap langkah pejuang-pejuang yang berjiwa ksatria dan pantang menyerah terus berjuang bersama bersatu berhimpun dalam menghadapi musuh-musuh pengecut yang akan terpecut oleh keberanian dan keinginan untuk memperluas daerah kebengisannya menuju pusat ekonomi yang maju dari robohan kehancuran peradaban yang lemah. Awal mula ini menjadi momentum bagi kita semua untuk mengapresiasi bagaimana persiapan kami dalam meneteskan pengaruh sang Dewa Perang di kehidupan modern ini merupakan fondasi dalam pembentukan dunia yang terstruktur, berdaulat, adil, dan makmur.

Chapter 3

Framebuffer, Interrupt, Driver

3.1 Driver Text Framebuffer

Framebuffer essentially adalah region di memory yang menyimpan bit-bit yang basically akan ditunjukkan ke Interface User. Pada tubes ini, by default kita pakai VGA3 which supports resolusi karakter ASCII 80x25. Mekanisme komunikasinya ada Memory-Mapped I/O yang membaca dan mengedit memori di lokasi tertentu serta Port-Mapped I/O melewati port pake ASM.

3.2 framebuffer.h dan framebuffer.c

Constants: `FRAMEBUFFER_MEMORY_OFFSET` basically marks awal dari terminal framebuffer. `CURSOR_PORT_CMD` dan `CURSOR_PORT_DATA` is magic.

3.2.1 framebuffer_write

Menuliskan karakter ke framebuffer menggunakan 2 bytes memori. Memori genap memuat karakter ASCII, sedangkan memori ganjil membawa informasi warna background dan warna teks.

Karena framebuffer bisa direpresentasikan sebagai matriks (array of array), kita bisa menentukan posisi yang kita cari dengan menggunakan

$$position = (y_position * characters_per_line) + x_position;$$

Jangan lupakan orang ketiga yaitu aritmatika pointer.

Diambil langsung dari OSDev, untuk coloring kayak gini.

```
Bit 76543210
    ||||||||
    |||||^^^-fore colour
    ||||^----fore colour bright bit
    |^^^-----back colour
    ^--------back colour bright bit OR enables blinking Text
```

8 bit upper di atas adalah buat coloringnya (background color dan text color). 8 bit seterusnya adalah ascii character-nya.

Implementasi (might change kalo udah ngerjain bonus):

```
void framebuffer_write(uint8_t row, uint8_t col, char c,
    uint8_t fg, uint8_t bg) {
    if (row >= 25 || col >= 80)
        return;

    uint32_t offset = ((uint32_t)(row * 80) + col) * 2;
    FRAMEBUFFER_MEMORY_OFFSET[offset]     = (uint8_t)c;
    FRAMEBUFFER_MEMORY_OFFSET[offset + 1] = (uint8_t)(((bg &
        0x0F) << 4) | (fg & 0x0F));
}
```

Ada edge case handling in case sudah mencapai limit x dan y dari si framebuffer ini. Offset menghitung basically posisi sel berdasarkan rumus sebelumnya dan dikalikan dua. Alasan dikalikan dua adalah karena tentu ukuran char framebuffer kita 2 bytes (ini bukan aritmetika pointer sih soalnya Bagas akhirnya pake offset which is just regular unsigned 32 bit integer). Terus akses offset awal (+ 1 byte) dan ubah jadi si char input, dilanjutin offset + 1 (another 1 byte) yang digabungkan dengan bitwise or (|). Di sini ada juga 0x0F buat mencegah si bg dan fg each lebih dari 4 bits.

3.2.2 framebuffer_set_cursor

Basically set kursor teks ke posisi tertentu. Di sini, harus pake port menggunakan instruksi assembly in/out yang disediain di portio.c (ini dijelasin lagi nanti).

Di text mode, kursor hanyalah daerah kedap-kedip (blinking) yang bisa dimanipulasi di OS.

Implementasi:

```
void framebuffer_set_cursor(uint8_t r, uint8_t c) {
    uint16_t pos = (uint16_t)((r * 80) + c);

    // Send high byte of cursor position to VGA CRT
        controller
    out(CURSOR_PORT_CMD, 0x0E);
    out(CURSOR_PORT_DATA, (uint8_t)((pos >> 8) & 0xFF));

    // Send low byte of cursor position to VGA CRT controller
    out(CURSOR_PORT_CMD, 0x0F);
    out(CURSOR_PORT_DATA, (uint8_t)(pos & 0xFF));
```

```
}
```

Kita menggunakan pos untuk menyatakan posisi kursor dengan penentuan rows dan cols, yaa udah tau lah ya. Kemudian kirim high byte dan low byte dengan metode yang tertulis. Baca sendiri.

3.2.3 framebuffer_clear

Sesuai namanya, untuk membersihkan framebuffer.

Implementasi:

```
void framebuffer_clear(void) {
    for (uint8_t row = 0; row < 25; row++) {
        for (uint8_t col = 0; col < 80; col++) {
            framebuffer_write(row, col, ' ', 0x07, 0x00);
        }
    }
    framebuffer_set_cursor(0, 0);
}
```

Mengisi semua rows dan columns dengan elemen kosong ' ', forecolor-nya 0x07 (gray) dan backcolor-nya 0x00 (black). Kursor juga direset ke posisi semula.

3.3 Interrupt

IIIIINTERUPSI ARESSSSSS!

Di OS yang kita buat, CPU hanya bisa menjalankan 1 task at a time. Oleh karena itu, perlu interrupt yang memberi semacam notifikasi agar CPU bisa melakukan event lain yang butuh perhatian. Pada bab ini, kita fokus di Hardware Interrupt / Interrupt Request (IRQ) yang sesuai namanya, dibuat oleh hardware, yang dalam konteks ini adalah keyboard. Suffix interrupt dinyatakan dengan "INT xh" dengan x adalah nomornya.

3.3.1 Sedikit info

PIC atau Programmable Interrupt Controller berfungsi mengontrol IRQ dari komponen eksternal. Pada kasus ini, akan dilakukan remapping ulang, karena suatu masalah, yaitu nomor interrupt oleh PIC dengan x86 CPU Exception (Software Interrupt i think) kurleb sama. Jadi diubah dikit biar memudahkan. PIC by default menyalakan IRQ0 (INT 21h) which is timer

3.3.2 interrupt.h

Ada BANYAK constants, kita ignore aja karena mostly buat PIC remapping. Ada juga CPURegister, yaa menyimpan representasi-representasi register. Ada InterruptStack yang merupakan data yang di-push ke stack ketika ada interrupt/exception. Interrupt frame whichis state CPU sebelum interrupt and also Interrupt Vector (identifier). Harusnya dibahas di chapter lain?

3.3.3 interrupt.c

io_wait untuk sinkronisasi 1-4 ms port I/O. Semua IRQ yang di-generate oleh PIC kita harus di-ACK (acknowledge?).

pic_remap untuk remapping PIC-nya. main_interrupt_handler sebagai Interrupt Handler atau Interrupt Service Routine (ISR) yang akan dipanggil ketika CPU menerima Interrupt.

activate_keyboard_interrupt akan dijelasin di bagian keyboard.

Banyak isinya dan magic semua jadi kita anggap udah baca aja.

3.4 Interrupt Descriptor Table

Disebut juga IDT, basically implementasinya mirip GDT. Jadi naturally ada InterruptDescriptorTable, IDTR, dan yang paling beda IDTGate (technically 11/12 sama GDT dan SegmentDescriptor)

Di IDTGate, ada... baca sendiri. Yaa harusnya ga jauh beda dari SegmentDescriptor.

3.4.1 idt.c

```
void initialize_idt(void) {
    // TODO : Implement (done)
    for (int i = 0; i < ISR_STUB_TABLE_LIMIT; i++) {
        set_interrupt_gate(i, isr_stub_table[i],
            GDT_KERNEL_CODE_SEGMENT_SELECTOR, 0);
    }
    __asm__ volatile("lidt␣%0" : : "m"(_idt_idtr));
    __asm__ volatile("sti");
}
```

Melakukan set_interrupt_gate di semua Interrupt Handler di isr_stub_table pada segmen kode kernel GDT, dengan privilege 0. Kemudian masukin ke assembly si IDT ini dan diaktifasi dengan Sistem dan Teknologi Informasi.

```
void set_interrupt_gate(
    uint8_t  int_vector,
    void     *handler_address,
    uint16_t gdt_seg_selector,
    uint8_t  privilege
) {
    struct IDTGate *idt_int_gate =
       interrupt_descriptor_table.table[int_vector];
    // TODO : Set handler offset, privilege & segment (done)
    // 32-bit handler address split into 2 x 16-bit (
       offset_low & offset_high)
    idt_int_gate->offset_low  = (uint32_t) handler_address &
       0xFFFF;
    idt_int_gate->offset_high = ((uint32_t) handler_address
       >> 16) & 0xFFFF;
    idt_int_gate->segment     = gdt_seg_selector;
    idt_int_gate->privilege   = privilege;
    idt_int_gate->_reserved   = 0;

    // Target system 32-bit and flag this as valid interrupt
       gate
    idt_int_gate->_r_bit_1    = INTERRUPT_GATE_R_BIT_1;
    idt_int_gate->_r_bit_2    = INTERRUPT_GATE_R_BIT_2;
    idt_int_gate->_r_bit_3    = INTERRUPT_GATE_R_BIT_3;
    idt_int_gate->gate_32     = 1;
    idt_int_gate->valid_bit   = 1;
}
```

Basically membuat suatu Interrupt Gate dengan vektor yang sudah ditentukan. offset_low menyimpan handler_address, sedangkan offset_high dilakukan right-shift sebanyak 16 bit. Sisanya menyesuaikan saja

3.5 Keyboard

Keyboard menggunakan IRQ1 (INT 21h). Melanjutkan diskusi terkait activate_keyboard_interrupt, fungsinya adalah mengaktivasi interrupt keyboard melalui port. Setelah menyala, setiap kali keyboard ditekan, keyboard akan mengirim IRQ1 ke CPU, masuk main_interrupt_handler, lalu memanggilan keyboard_isr. Kita perlu device driver untuk mengabstraksikan proses dari ngetik keyboard ke tampil di layar. Di sini, kita menggunakan scancode dari keyboard yang nantinya akan di-map ke ASCII.

3.5.1 Keyboard ISR

Interrupt Service Routine-nya keyboard. Basically bagian utama driver keyboard. ISR akan menerima scancode yang dimasukkan oleh keyboard. Jika state keyboard_state.keyboard_input_on bernilai true, ISR melakukan mapping scancode ke ASCII menggunakan tabel keyboard_scancode_1_to_ascii_map, lalu menyimpan karakter ASCII ke buffer ke keyboard_state.keyboard_buffer.

Implementasi:

```
void keyboard_state_activate(void) {
    keyboard_state.keyboard_input_on = true;
}

void keyboard_state_deactivate(void) {
    keyboard_state.keyboard_input_on = false;
}
```

Fungsi aktivasi dan deaktivasi ISR keyboard. Apakah interrupt keyboard didengarkan atau tidak.

```
void keyboard_isr(void) {
    uint8_t scancode = in(KEYBOARD_DATA_PORT);

    if (keyboard_state.keyboard_input_on && scancode < 0x80)
       {
        char mapped_char = keyboard_scancode_1_to_ascii_map[
           scancode];
        if (mapped_char != 0)
            keyboard_state.keyboard_buffer = mapped_char;
    }

    pic_ack(IRQ_KEYBOARD);
}
```

Sesuai yang dijelaskan, receive interrupt, map char, safe to buffer, also janlup pic_ack.

```
void get_keyboard_buffer(char *buf) {
    if (buf != (void *) 0) {
        *buf = keyboard_state.keyboard_buffer;
        keyboard_state.keyboard_buffer = 0;
    }
}
```

Mengambil char di keyboard_state.keyboard_buffer, lalu di-flush (dihapus). Ini buat nanti pas kita write dari keyboard.

3.5.2 Tambahan

Ketika kita mengetik 1 key, sebenarnya ada dua peristiwa yang terjadi, yaitu make and break. Make Code adalah scancode yang dihasilkan saat kita tepat mengetik, sedangkan Break Code saat kita melepasnya. Di tubes ini, kita memakai scancode set 1 PS/2 yang terima jadi aja. Break Code hanyalah Make Code yang bit teratas-nya dijadikan 1. First half map-nya adalah area untuk Make Code, sedangkan second half-nya untuk Break Code.

Untuk extended key opsional dan sejauh ini belum diimplementasi. Extended key misalnya adalah arrow keys, shift, caps lock, yang saat ini nilainya nol di map kita dan posisinya di first half map-nya yaitu di area Make. Jika ingin diimplementasi, manfaatin keyboard_state.read_extended_mode dan lakukan manipulasi lebih lanjut.

3.6 Koordinasi yang Taktis dan Keberanian yang Mumpuni

Awan Cumulonimbus muncul di langit dan meraung-raung, entah konsekuensi dari pemberontakan kami atau hanya kebetulan saja, melihat kerja keras Sang 4 Raja dalam upaya resureksi Ares serta insurgensi terhadap peradaban ini. Ketika hujan badai dan hail representasi simbolik Zeus turun ke daratan manusia yang mudah hancur, kami menganggap hal tersebut sebagai motivasi lebih lanjut dalam melanjutkan progres kami yang tak akan berhenti sampai dunia ini dihancurkan oleh kebodohan itu sendiri. Dengan hati yang lunak, kami mengisi ketiadaan Ares dengan keberanian yang didasari oleh dimensi struktural yang lebih kuat dari baja dan berlian yang dipertulangkan. ”Interupsi, Ares!” menjadi sebuah pemantik yang kami rasa cukup bagi para manusia biasa pada saat mereka ingin berkomunikasi dengan pemimpin baru mereka. Keberanian yang akan mereka rasakan tidak akan sepadan dengan apa yang dirasakan para SPARTANS pada era tersebut, namun, dengan koordinasi yang taktis, kami rasa hal yang Ia nanti-nanti suatu saat akan terbentuk juga. Dunia baru akan terbentuk, dan segala masukan dan keluaran yang muncul dari umat manusia akan menjadi suatu bentuk kontribusi yang nyata dengan visualisasi yang terbukti ketika hati mereka terbentuk sebagai batu yang tak terpecahkan nan pikiran dengan keenceran AIR yang melimpah.

Chapter 4

Filesystem

4.1 Disk Drivers

Pada bagian ini, diimplementasi driver disk ATA (Advanced Technology Attachment) PIO. Satuan data utama yang akan digunakan adalah block, yang akan dijelaskan lebih lanjut di bagian kemudian. Terdapat penambahan pada `portio.c` berupa instruksi `in16` dan `out16`, which is the same instructions yang digunakan untuk menerima dan mengirim data melalui port, tapi kali ini 16-bit untuk memudahkan (katanya). Implementasi ini menggunakan logical block addressing yang menyatakan indeks block.

4.2 disk.h dan disk.c

Pada disk.h, ada beberapa hal yang menarik. Perhatikan bahwa `BLOCK_SIZE` didefinisikan sebagai 512 (baca bytes). Terdapat pula tipe data BlockBuffer sebagai representasi dari sebuah block yang terdiri dari byte buffer berukuran `BLOCK_SIZE`.

Pada disk.c, diimplementasikan beberapa fungsi yang kaitannya dengan ATA busy waiting, DRQ waiting, send commands, cukup tahu saja. Dagingnya di bagian sini adalah `read_blocks()` dan `write_blocks()`. Implementasi cukup tahu aja. Fungsi `read_blocks()`:

```
void read_blocks(void *ptr, uint32_t
    logical_block_address, uint8_t block_count);
```

berperan mengambil data pada logical block address tertentu sebanyak block count dan menyimpannya pada pointer.

Fungsi `write_blocks()` juga kurang lebih mirip:

```
void write_blocks(void *ptr, uint32_t
    logical_block_address, uint8_t block_count);
```

berperan menulis data yang berada di pointer ke logical block address sebanyak block count.

Tambahan juga bahwa karena block berukuran 512 bytes, LBA pertama dimulai dari memory 0 hingga 511 (karena zero-indexed) dari 0x0 hingga 0x1FF, LBA kedua

dari 512 sampai 1023, (0x200 hingga 0x3FF), dst.

4.3 The Source of Every Filesystem Knowledge

Note: Bagian ini fokus ke materi.

Block adalah unit dasar penyimpanan dalam filesystem. Pada filesystem ini, ukuran block adalah 512 bytes. Segala hal essentially disimpan di dalam block. Lalu, bagaimana block yang sudah dipakai di-track? Caranya adalah menggunakan bitmap block. Inode adalah struktur data yang menyimpan metadata terkait suatu file (lebih lengkap di bagian Inode sendiri). Inode juga ada bitmapnya. Untuk definisi-definisi lain, rasanya cukup sama dengan di modul saja. Ada superblock yang menyimpan informasi global tentang block-block dan inode-inode, ada block group yang terdiri dari beberapa block.

Block group memuat superblock (group pertama saja), group descriptor, block bitmap, inode bitmap, inode table, dan data blocks. Layoutnya kurang lebih seperti ini

```
block 0         boot sector (legacy)
block 1         superblock              (44 byte terpakai)
block 2         BGD table               (256 byte terpakai)
block 3         block bitmap grup 0
block 4         inode bitmap grup 0
block 5..20     inode table grup 0  (16 block)
block 21..      data blocks grup 0
block 1024..    grup 1, dan seterusnya (8 grup x 1024 block =
   8192 block)
```

(ini dibuat kakas jadi koreksi lagi aja, ya secara garis besar mirip-mirip lah)

4.3.1 Filesystem Constants

Pertama-tama, kita berkenalan dengan konstanta-konstantanya terlebih dahulu. Boot sector berada pada block ke-0, yang di kasus ini

Konstanta	Nilai	Penjelasan
BOOT_SECTOR	0	Indeks block BOOT_SECTOR. fs_signature akan dimasukkan ke sini.
DISK_SPACE	4194304 bytes	Disk space ukuran 4 MiB. Sama Dip diganti jadi 8 kyknya
INODE_SIZE	70 bytes	sizeof(EXT2Inode)
INODES_PER_TABLE	7 inode per block	(BLOCK_SIZE / INODE_SIZE)
GROUPS_COUNT	8 group	BLOCK_SIZE / sizeof(struct EXT2BlockGroupDescriptor) / 2u
BLOCKS_PER_GROUP	1024 block	(DISK_SPACE / BLOCK_SIZE) / GROUPS_COUNT
INODES_TABLE_BLOCK_COUNT	16 block	Jumlah tabel inode dalam satu group
INODES_PER_GROUP	112 inode	INODES_PER_TABLE * INODES_TABLE_BLOCK_COUNT
ROOT_INODE	2	Inode nomor 2 sebagai root
DIRECT_BLOCKS	12	Direct pointer pada i_block
POINTERS_PER_BLOCK	64	Jumlah pointer block dalam indirect block

4.3.2 Superblock, BGD, BGDT

Seperti yang telah dijelaskan sebelumnya, Superblock (EXT2Superblock) memuat data-data global terkait semua block seperti jumlah inode, jumlah block, nomor root inode, jumlah block free, jumlah inode free, dll. Isinya pretty straightforward berdasarkan yang ada di header.

Block Group Descriptor (EXT2BlockGroupDescriptor) menyimpan atribut-atribut seperti id bitmap dan inode table group tersebut (doesn't actually store the bitmaps and inode tables, tapi nunjuk ke block pada group yang menyimpan ybs). BGD juga nyimpen free blocks count, free inodes count, jumlah inode yang digunakan untuk directory, dll. BGD disimpan dalam BGDT, sebuah tabel yang menyimpan sejumlah `GROUPS_COUNT` BGD. Di InterupsiArOS, table tersebut hanya disimpan di group 0, tapi tetap mewakili setiap group obv.

4.3.3 Bitmap

Bitmap berukuran 1 block dan memuat data mengenai apakah suatu block/inode pada group terpakai atau tidak. Jika terpakai nilainya 1, jika tidak 0. Dalam block 512 byte, 1 bit mewakili item ke-n.

4.3.4 Inode dan Inode Table

Inode (EXT2Inode) pada sebuah group memiliki posisi dimulai dari tabel inode group tersebut. Inode dikenali dengan nomornya (misal root inode adalah 2).

Member	Penjelasan
i_mode	Filetype dan access right
i_size	Ukuran file (bytes)
i_blocks	UKURAN blocks yang dipakai
i_block[]	block yang mengandung data yang ditunjuk inode ini

`i_mode` akan bernilai `EXT2_S_IFREG` 0x8000 jika regular file dan `EXT2_S_IFDIR` 0x4000 jika adalah directory.

Inode menggunakan direct block pada 12 pointer pertama yang menunjuk langsung ke block; pointer ke-13 menggunakan indirect block menunjuk ke block yang menunjuk ke direct block; 14 doubly-indirect; 15 triply-indirect.

Inode Table adalah tabel berii Inode berjumlah `INODES_PER_GROUP`.

4.3.5 Directory Entry

Bagian ini cukup abstrak bahkan kami saat ini masih bingung bagaimana menjelaskannya. Ibarat kata directory entry (direntry) adalah di mana kita bisa menemukan sebuah inode jika kita memiliki namanya. Walaupun namanya directory entry, file yang bukan directory juga memiliki ini. Perhatikan juga bahwa selalu ada . dan .. di setiap directory

Jenis file type ada juga

```
#define EXT2_FT_UNKNOWN 0 // Unknown File Type
#define EXT2_FT_REG_FILE 1 // Regular File
#define EXT2_FT_DIR 2 // Directory
#define EXT2_FT_NEXT 3 // Character Special File
```

Member	Penjelasan
inode	Nomor inode yang merepresentasikan entry, 0 jika unused
rec_len	jarak ke next entry (essentially isi dari struct)
name_len	panjang nama
file_type	file type

BUTBUTBUTBUT. Di luar DirectoryEntry itu ada name dan padding supaya kelipatan 4.

4.4 Daging

Penjelasan TIAP kode ext2.c yang jumlah halamannya hampir 1000 itu. Note: Sepertinya waste of space dan overdocumented kalo copas semuanya, jadi silakan compare saja dengan yang di source code. Note 2: Beberapa penjelasan akan JAUH lebih panjang daripada penjelasan lain.

Sebelumnya, ada tambahan lagi constant dan static variables. Constant-nya seperti `GROUP0_FIRST_META` yang merupakan indeks block pada group setelah boot sector, superblock, dan tabel bgd. Ada juga `ROOT_INODE` yang kami pilih jadi 2 (1 reserved). Ada juga `GROUP_META_BLOCKS` which is offset indeks block setelah block bitmap, inode bitmap, dan 16 inode table tadi.

```
char *get_entry_name(void *entry);
```

Penjelasan: Memperoleh POSISI nama directory entry dengan menambahkan address entry dengan `sizeof(EXT2DirectoryEntry)` which is headernya.

```
uint16_t get_entry_record_len(uint8_t name_len);
```

Penjelasan: Memperoleh `rec_len` sebuah entry dengan menambahkan ukuran header dan panjang nama. Dibulatkan ke atas menuju kelipatan 4

```
uint32_t inode_to_bgd(uint32_t inode);
```

Penjelasan: Inode berada di group berapa? Note nomor inode tidak zero-indexed jadi harus dikurangi satu dulu. Berlaku untuk seterusnya

```
uint32_t inode_to_local(uint32_t inode)
```

Penjelasan: Inode berada di indeks ke-berapa dalam group? Obv zero-indexed

```
void init_directory_table(struct EXT2Inode *node, uint32_t
    inode, uint32_t parent_inode);
```

Penjelasan: Mengisi directory baru pada node dengan '.' (inode) dan '..' (parent inode). Perhatikan penggunaan buffer untuk memudahkan manipulasi memori dan operasi memori seperti memcpy, memset (kalo diisi 0 biar ngilangin garbage data), dll untuk kedepannya juga. Directory entry untuk self dan parent disusun, note yang parent ukurannya 500 soalnya harus ngisi block. Kemudian, isi data di node dan write ke block.

```
bool is_empty_storage(void);
```

Penjelasan: Melakukan memcmp fs signature dengan block di BOOT SECTOR.

```
static void set_bit(struct BlockBuffer *bitmap, uint32_t
    index)
```

Penjelasan: Menandai bit ke-n pada bitmap sebagai used. Hanya bisa per byte, makanya formulasinya gitu.

```
void create_ext2(void);
```

Penjelasan: Perlu diketahui block 1 itu perlu kita set signature, superblock, bgd table, dan juga data block pertama. Jadi logicnya kelihatan lah ya harusnya buat block 0 bakal beda sendiri

1. Set up block zero (nol semua), ini buat bersihin inode table nanti; tulis fs signature ke block 0 (boot sector); tentukan root block yaitu indeks data block setelah metadata pada group 0.

2. Loop for all groups (0 s.d. 7)

3. Untuk group 0, basenya 3 dan used blocks 3 + 18 + 1 (dari root block directory) jadi 22; untuk group 1-7 basenya g * blocks per group, used blocks cukup 18 dari group meta blocks aja.

4. Isi bgd-nya. Standar aja sih, bitmap offset, table offset, free blocks count, free inodes count (includes root inode sama reserved), used dirs satu karena root directory

5. Pake block buffer bersih, set bit sebanyak used blocks, lalu write ke bitmap block
6. Pake block buffer bersih, set bit di inode 1 (index 0) dan di root inode, lalu write ke bitmap inode
7. Bersihkan inode table pakai block zero
8. Buat buffer bersih, copy isi group descriptor table ke buffer, lalu write ke LBA 2
9. Isi-isi superblock, standar lah. Write ke LBA 1.
10. Kita udah taro signature, superblock, bgdt. Sekarang obviously perlu ada inode yang refer to root directory. Jadi buat itu lah, standar standar aja.
11. Inode udah ada, tapi belom ditaroh infonya di inode table. Ambil table block dengan offsetnya, trus write di table block. Alasan ada kosongan di buffer adalah untuk reserved

```
static bool is_superblock_valid(void);
```

Penjelasan: Generic checking aja, #kurakura#

```
void initialize_filesystem_ext2(void);
```

Penjelasan: Cek apakah storage empty. Jika empty maka create ext2. Load block 1 ke superblock static,cek superblock apakah valid, jika tidak sama juga create ext2. Load block 2 ke gdt static. Hitung ulang free counters just in case ada inkonsistensi

Tambahan: `MAX_NODE_BLOCKS` adalah jumlah maksimal data block yang bisa dialamatkan satu inode (direct block + single indirect + double indirect). Triply indirect gada karena yaa ga perlu juga ukuran disknya cukup kecil. Struct `BlockCursor` basically sebagai penunjuk kita udah di mana saat kita iterate di blocks-nya, ada `buf` (posisi di buffer sekarang), `bytes_left` (sisa byte), dan `blocks_left` (sisa data block).

```
static bool test_bit(struct BlockBuffer *bitmap, uint32_t
    index);
```

Penjelasan: Mirip set bit, tapi ngecek apakah bit ke-n pada bitmap bernilai 1 (used).

```
static void clear_bit(struct BlockBuffer *bitmap, uint32_t
    index);
```

Penjelasan: Mirip set bit dan test bit, tapi ya sesuai namanya di-unset

```
static uint32_t inode_table_block(uint32_t inode);
```

Penjelasan: Menentukan inode berada di block ke-berapa dalam suatu group.

```
static uint32_t inode_table_offset(uint32_t inode);
```

Penjelasan: Offset bytes inode dari awal block.

```
static bool is_inode_allocated(uint32_t inode);
```

Penjelasan: Kalo nomor inode 0 atau melebihi jumlah total inode langsung false. Otherwise baca inode bitmap block ke block buffer, lalu `test_bit`.

```
struct EXT2DirectoryEntry *get_directory_entry(void *ptr,
    uint32_t offset);
```

Penjelasan: Mencari entry pada ptr + offset. ptr di-cast ke (uint8_t*) supaya offset-nya per bytes.

```
struct EXT2DirectoryEntry *get_next_directory_entry(struct
    EXT2DirectoryEntry *entry);
```

Penjelasan: Mencari directory entry selanjutnya dengan menggunakan get directory entry dengan offset rec len entry itu sendiri.

```
uint32_t get_dir_first_child_offset(void *ptr);
```

Penjelasan: Mencari offset child pertama sebuah directory. Karena udah ada '.' dan '..', jadi ambil dari offset rec len self dan parent.

```
static bool is_entry_in_block(struct BlockBuffer *block,
    uint32_t offset);
```

Penjelasan: Mengecek apakah ada entry yang valid di offset block tersebut. Kalau offset dan header melebihi block size return false (krn invalid). Lakukan get directory entry pada block dengan offset dan cek alignment, ukuran, etc. valid.

```
static bool is_block_empty(struct BlockBuffer *block);
```

Penjelasan: Cek semua entry di block, kalo ada yang inode-nya gak 0, false.

```
static bool is_entry_name(struct EXT2DirectoryEntry *entry,
    const char *name, uint8_t name_len);
```

Penjelasan: Mengecek apakah entry ini punya nama yang dicari. Inode harus terpakai (non zero), panjang nama sama, dan isi namanya sama.

```
static bool is_dot_entry(const char *name, uint8_t name_len);
```

Penjelasan: Mengecek apakah namanya '.' atau '..'

```
static void fill_directory_entry(struct EXT2DirectoryEntry *
    entry, uint32_t inode, const char *name, uint8_t name_len,
     uint8_t file_type);
```

Penjelasan: Mengisi inode, panjang nama, file type, dan nama pada entry, lalu sisa ruang sampai akhir rec len dibuat 0 biar clean. Perhatikan rec len ga di-update di sini.

```
void load_node(struct EXT2Inode *node, uint32_t inode);
```

Penjelasan: Baca block inode table yang memuat inode, lalu copy inode di offset-nya ke node.

```
void sync_node(struct EXT2Inode *node, uint32_t inode);
```

Penjelasan: Kayak yang bagian akhir create ext2. Essentially write node ke inode table-nya, tapi jangan sampai overwrite inode yang lain di block. Caranya dengan nyimpen isi block ke block buffer, terus ngisi offset buffer dengan node, terus di-write

```
void commit_metadata(void);
```

Penjelasan: Tulis superblock dan BGDT ke disk dari memory. Perlu ada karena yaa ofc you're gonna save the metadata after doing CRUD operations.

```
uint32_t allocate_node(bool is_directory);
```

Penjelasan: Cari inode free di semua group dengan mengecek apakah tersedia lewat bitmap-nya (first fit oooooo). Kalo ketemu, set bit buat bitmap, write bitmap group, kurangi free inode di bgd dan superblock. Mengembalikan 0 kalo udah gaada lagi yg unallocated. Kalo tersedia, return nomor inode keberapa.

```
void deallocate_node(uint32_t inode);
```

Penjelasan: Kalo root inode dan inode yang belum dialokasikan ignore. Load node dari disk, free blocks pake deallocate blocks (dijelasin nanti), baca bitmap dan clear bit pada inode bitmap also write, tambah free counter, kalo node directory tambahin dirs count, trus nol-kan node pada inode table.

```
uint32_t allocate_block(uint32_t prefered_bgd);
```

Penjelasan: Mirip allocate node tapi yaa buat block dan berdasarkan berdasarkan block mulai dari mana, dan bisa muter juga. Return block kalau ga penuh, 0 kalo penuh.

```
void deallocate_block(uint32_t block);
```

Penjelasan: Sesuai namanya deallocate block, kalo block 0 atau di luar range ignore, kalo di bitmap-nya belom allocated juga ignore. Sisanya standar-standar lah.

```
static void recount_free_counters(void);
```

Penjelasan: Hitung ulang free blocks dan free inodes tiap group dari bitmap, simpan ke BGD dan superblock. Fungsinya kalo misal ada operasi yang belum sempat melakukan commit metadata.

```
static uint32_t count_free_blocks(void);
```

Penjelasan: Menghitung free blocks tapi dari bitmap-nya.

```
static uint32_t count_pointer_blocks(uint32_t blocks);
```

Penjelasan: Menghitung berapa pointer yang dibutuhkan untuk menyimpan jumlah data block tertentu Kalo sampai 12 block, 0 karena direct semua. Kalo dikurangi 12 masih di bawah pointers per block, jadi 1 (single indirect). Kalo misal dikurangi pointers ke block masih ada, jadi double indirect sehingga 2 + ceil(blocks / pointers per block). Kalo kata claude

1. 1 block single indirect (sudah terisi penuh, tetap dihitung),
2. 1 block puncak double indirect,
3. 1 block tingkat dua untuk setiap 128 data yang tersisa.

```
static uint32_t write_block_tree(struct BlockCursor *cursor,
    uint32_t depth, uint32_t prefered_bgd);
```

Penjelasan: Write block secara tree recursive berdasarkan depth-nya. Pakai bantuan BlockCursor.

1. Alokasikan satu block untuk node tree ini.
2. Kalau depth-nya 0, ambil size at most 1 block at least bytes left dari cursor. Move buffer cursor-nya sebanyak size, kurangin bytes left dan blocks left, terus write ke block. Kalo misal buffer di BlockCursor null tulis nol di block tersebut (ini buat directory) dan return nomornya.
3. Kalau depth lebih dari 0, siapin array pointer dan isi pake write block tree dengan depth-1, sampai sebanyak pointers per block atau blocks left di cursor sudah habis, terus write.

```
static void read_block_tree(struct BlockCursor *cursor,
    uint32_t block, uint32_t depth);
```

Penjelasan: Kyk write block tree tapi read. Kalo depth 0, baca data block dan copy ke cursor buffer. Atribut cursor atur-atur. Perhitungan rekursi kalo depth > 0 kurang lebih sama sih.

```
static void deallocate_block_tree(uint32_t block, uint32_t
    depth, uint32_t *blocks_left);
```

Penjelasan: Kyk write and read block tree tapi buat dealloc block instead. Kalo depth 0, cukup kurangin blocks left. Kalo nggak yang penting bukan block 0, itu di read blocks dulu trus deallocate block tree secara rekursif. Di akhir, secara umum dipake deallocate block.

```
void deallocate_blocks(void *loc, uint32_t blocks);
```

Penjelasan: Deallocate block sebanyak blocks yang ingin di-free pada suatu node blocks (loc adalah node->i_block). Apparently di-copy dulu dari loc ke array location supaya 4-byte aligned.

```
bool allocate_node_blocks(void *ptr, struct EXT2Inode *node,
    uint32_t prefered_bgd);
```

Penjelasan: Alokasiin blocks berdasarkan node yang member-nya udh terdefinisi.

1. Return false kalo jumlah block melebihi MAX NODE BLOCKS atau data block + pointer block lebih banyak dari free. Kalo gagal ada yang teralokasi, ga butuh rollback
2. Buat cursor berdasarkan yang didefinisikan di node, set i_block as 0 dulu.
3. Isi 12 direct block dengan write block tree depth 0.
4. Kalo perlu indirect block, write block tree rekursif juga.

Masih harus sync node biar kesave ke disk

```
void read_node_blocks(void *ptr, struct EXT2Inode *node);
```

Penjelasan: Mirip allocate node blocks, tapi buat read node blocks.

```
bool is_directory_empty(uint32_t inode);
```

Penjelasan: Iterate semua block (direct only) inode dan cek semua entry-nya. Kalau inode di-entrynya ga ketemu nol langsung false. Offset awal udh mempertimbangkan '.' dan '..' pake get dir first child offset.

```
bool load_directory_node(struct EXT2Inode *node, uint32_t
    inode);
```

Penjelasan: Load inode ke node. Validasi sebelumnya kalo udh/belom dialokasi, terus dicek juga apakah i mode nya directory (EXT_S_IFDIR), i blocksnya lebih dari 0 kah, i block 0 tidak 0 kah.

```
uint32_t find_directory_entry(uint32_t dir_inode, const char
    *name, uint8_t name_len);
```

Penjelasan: Mencari folder dengan nama tertentu di dalam directory. Load dir inode kemudian iterate untuk tiap direct blocks-nya, lakukan pencarian tiap entry dan return inode-nya kalo entry name dan file type entry-nya sesuai (`EXT2_FT_DIR`)

```
int32_t get_file_type(uint32_t dir_inode, const char *name,
    uint8_t name_len);
```

Penjelasan: Mirip find directory entry, tapi return file-type jika sesuai namanya.

```
uint32_t find_file_entry(uint32_t dir_inode, const char *name
    , uint8_t name_len);
```

Penjelasan: Mirip find directory entry, tapi nyari entry yang punya tipe file alias yang kodenya (`EXT2_FT_REG`)

```
bool add_directory_entry(uint32_t dir_inode, uint32_t inode,
    const char *name, uint8_t name_len, uint8_t file_type);
```

Penjelasan: Menambahkan entry baru ke directory. Suatu entry bisa punya rec len yang lebih besar dari name len-nya, dan sisa ruang di ekornya untuk entry yang baru. Note: cuma direct blocks aja

1. Hitung ukuran entry baru pakai get entry record len (needed), terus load directory inode.
2. Cek tiap direct block dan entry di dalamnya. Hitung used yaitu ukuran rec len entry tersebut (0 kalo unallocated).
3. Kalo misal rec len entry >= jumlah ruang used + needed, berarti muat. Kalo muat dan used ga 0, entry lama dipotong jadi seukuran used, sisanya jadi entry baru after entry lama. Kalau used-nya 0, langsung pake entry tersebut.
4. Isi entry dengan fill directory entry, write block, dan return true.
5. Return false kalau memang melebihi direct blocks atau ga bisa allocate block lagi
6. Kalo emang ga ada ruang buat entry baru tapi bisa nambah block, kita bisa allocate block aja. Entry dibuat sepanjang block size, terus di-fill dan write. Jangan lupa sync node dan melakukan penyesuaian lain sewajarnya.

```
static void shrink_directory(struct EXT2Inode *dir, uint32_t
    dir_inode, uint32_t index);
```

Penjelasan: Free block directory pada index tertentu. Block terakhir dipindahkan ke block kosong. Sesuaikan atribut pada dir, lalu sync node. Ga perlu ngurut karena directory, bukan file reguler.

```
bool remove_directory_entry(uint32_t dir_inode, uint32_t
    target_inode, const char *name, uint8_t name_len);
```

Penjelasan: Sesuai namanya, untuk menghapus directory entry tertentu. Perlu target inode karena nama folder dan file bisa sama tapi harus bisa dibedain.

1. Kalau is dot entry ('.' atau '..') prevent penghapusan, lgsg return

2. Iterate tiap direct block dan entry di dalamnya, keep track of prev kyk linked list aja

3. Kalau entry sesuai target inode dan namanya bener, cek dulu prev-nya. Kalo prev-nya NULL alias ketemu pertama, set inode-nya as 0 aja. Kalo ngga, tambahin rec len-nya prev sama rec len-nya entry. Selebihnya write ke block inode dir.

4. Kalo bukan block pertama dan udah kosong, shrink directory (which removes blocks)

Seperti biasa, kalo berhasil return true; kalo gagal return false.

```
static bool is_folder_request(const struct EXT2DriverRequest
    *req);
```

Penjelasan: Cek apakah request memiliki buffer size 0 (syarat request directory).

```
static bool is_name_taken(uint32_t parent_inode, const char *
    name, uint8_t name_len, uint8_t file_type);
```

Penjelasan: Cek semua entry pada parent, lalu cek apakah ada yang nama dan tipe filenya sama.

```
static bool has_invalid_char(const char *name, uint8_t
    name_len);
```

Penjelasan: Nama ga boleh ada karakter / (buat separator shell) dan ga boleh pake end separator karena nanti kepotong pas jadi string.

```
static int8_t rollback_write(struct EXT2Inode *node, uint32_t
     inode);
```

Penjelasan: Cancel side effect dari write kalo gagal di tengah jalan. Kalo node ga NULL, sync node dulu karena deallocate node baca i mode dan i block dari disk. Deallocate node, commit metadata, return 1.

```
int8_t write(struct EXT2DriverRequest *request);
```

Penjelasan: Membuat file atau folder baru berdasarkan request.

1. Return -1 (unknown error) kalo request null, nama request-nya null, name len 0, atau has invalid char. Kalo file (not folder) tapi empty juga return -1.

2. Return 2 kalo parent ga valid pake load directory node.

3. Return 1 kalo namanya '.' atau '..' pake is dot entry atau kalo name taken sesuai file type-nya.

4. Alokasikan inode, kalo 0 (gada space lagi), return -1.

5. Kalo folder, alokasiin block dulu. Kalo misal ga ada (i block 0-nya 0), rollback write tapi sebelumnya i mode jadi directory dan i blocks set 0 dulu biar used dirs count di deallocate node dikurangin. Kalo emang berhasil, init directory table.
6. Kalo file, set i mode-nya as reg, i size sesuai buffer size, jumlah i blocks disesuaikan juga dan janlup ceil and make sure ga overflow. Allocate node blocks isi-nya request->buf, di node, pake bgd (whichis group-nya si inode). Kalo gagal rollback write.
7. Jangan lupa sync node.
8. Add directory entry yang bersesuaian ke parent inode. Kalo gagal, rollback node-nya null aja karena udah di-sync sebelumnya
9. Commit metadata, trus return 0

```
int8_t read(struct EXT2DriverRequest request);
```

Penjelasan: Membaca file ke request buf. Perhatikan `buf` NULL dianggap valid, karena program user di-load ke virtual address 0x0.

1. Return -1 kalo nama NULL or len name 0.
2. Return 4 kalo parent ga valid (load directory node failed)
3. Cari inode dengan find file entry. Kalo ga ada, cek file typenya. Kalo emang ga ketemu (file type -1), return 3. Kalo semisal dia bukan file (EXT2_FT_DIR or EXT2_FT_NEXT), bukan file berati return 1. Sisanya unknown filetype return -1.
4. Load node. Kalo buffer size request kurang dari size node-nya, return 2 (ga cukup).
5. Read node blocks ke request buf, return 0.

```
int8_t read_directory(struct EXT2DriverRequest *prequest);
```

Penjelasan: Mirip `read`, tapi yang dibaca isi directory-nya.

1. Return -1 kalo request NULL, buf NULL, nama NULL, atau panjang nama 0.
2. Return 3 kalo parent ga valid, stndar lah ye
3. Cari folder dengan find directory entry. Kalo ga ketemu, cek pake get file type. Return 2 kalo emg ga ada, return 1 kalp ternyata file, dan return -1 buat sisanya.

4. Load node. Return -1 kalo buffer size kurang dari node size.

5. Read node blocks terus return 0.

```
int8_t delete(struct EXT2DriverRequest request);
```

Penjelasan: Menghapus file atau folder, tapi kalo folder harus kosong. Pada request, `is_directory` dipake buat ngecek apakah yang dihapus adalah directory atau bukan.

1. Return -1 kalo nama NULL atau panjang nama 0.

2. Return 3 kalau parent ga valid.

3. Kalo directory, cari dengan find directory entry, return 1 kalo gak ketemu, dan return 2 kalo folder ga kosong. Return -1 kalo remove directory entry gagal (dot entry). Kalo berhasil semua, deallocate node.

4. Kalo file, find file entry. Kalo ga ketemu return 1, kalo ga bisa remove directory entry return -1 (dot entry). Kalo bisa deallocate node.

5. Commit metadata dan return 0

BAIIIIIKKKK. Itu saja untuk chapter 2 (sejauh ini)

4.5 Persiapan Atas Kemenangan

Kemenangan sudah setengah jalan di depan mata. Tidak ada yang dapat menghentikan kami, karena kami lah yang menentukan jalan kami sendiri. Tatkala umat manusia berpesta pora merayakan hari-hari ”kemenangan mereka”, bersembunyi di balik simfoni yang tampak megah nan ekstatik, kami mempersiapkan segudang arsip yang menyimpan segala jenis tulisan yang dapat digunakan oleh siapapun, baik kami ataupun umat manusia yang baru. Melalui sistem pengarsipan ini, segala dahaga akan pengetahuan sistem, teknologi, dan informasi akan dipenuhi melalui bekal-bekal yang telah dipersiapkan oleh para pendahulu: prajurit-prajurit SPARTA, petinggi-petinggi, para cendekiawan dari Latex V, bahkan rakyat jelata pada masa itu. Namun, janganlah lupakan bekal-bekal itu ketika kau datang kepada kami. Niscaya kami akan melakukan pengecekan terhadap spek tersebut, sebagai bentuk kesiapan kalian dalam berperang menghadapi kelemahan. Percayalah, wahai pembaca teks sakral ini: waktu yang kami habiskan tidak akan sia-sia. Ketika kau membaca buku ini, Ares telah kami berikan kendali atas dunia ini, dan bersyukurlah atas nikmat yang saat ini kau rasakan.

Chapter 5

References

Uhh biblatex ga bisa jadi pake gini aja
References we used:

- Tanya Gemini, Opencode, dan Claude
- Buku Keramat Intel
- `http://www.osdever.net/bkerndev/Docs/printing.htm`
- `https://wiki.osdev.org/Text_UI`
- `https://wiki.osdev.org/Text_Mode_Cursor`